



our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Stormrae

SECURITY REVIEW

Date: 2 February 2026

CONTENTS

1. About Shieldify	3
2. Disclaimer	3
3. About Stormrae	3
4. Risk classification	3
4.1 Impact	3
4.2 Likelihood	3
5. Security Review Summary	4
5.1 Protocol Summary	4
5.2 Scope	4
6. Findings Summary	5
7. Findings	6

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Stormrae

Stormrae is the participation layer for AI development. We turn all essential AI data tasks, adversarial testing, labeling, preference evaluation, and validation into engaging consumer experiences.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable - and covers vectors similar to grieving attacks that can be easily repaired

4.2 Likelihood

- **High** - almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** - still relatively likely, although only conditionally possible
- **Low** - requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 4 days with a total of 64 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one High and two Medium severity issues. They're mainly related to state management, specifically around PDA handling, counter mutability, and inconsistent on-chain state transitions leading to pricing issues and potential DoS.

The Stormrae team has done a great job and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Stormrae
Repository	kingarthur
Type of Project	AI Development Layer
Security Review Timeline	4 days
Review Commit Hash	0126608845630e1dac49f161a08a49ae934fae92
Fixes Review Commit Hash	5ee453592fd7f089f73fccb6838a1c28ce7fe7f1

5.2 Scope

The following smart contracts were in the scope of the security review:

File	Lines
programs/king-arthur-sc/src/admin/accept_winner.rs	99
programs/king-arthur-sc/src/admin/admin_transfer.rs	89
programs/king-arthur-sc/src/admin/cancel_winner.rs	199
programs/king-arthur-sc/src/admin/create_package.rs	89
programs/king-arthur-sc/src/admin/mod.rs	31
programs/king-arthur-sc/src/admin/pause_unpause_package.rs	75
programs/king-arthur-sc/src/admin/reset_contract.rs	47
programs/king-arthur-sc/src/admin/set_winner.rs	130
programs/king-arthur-sc/src/admin/update_max_winners.rs	59
programs/king-arthur-sc/src/admin/update_package.rs	78
programs/king-arthur-sc/src/admin/update_percentages.rs	37

programs/king-arthur-sc/src/admin/update_prize_admin.rs	23
programs/king-arthur-sc/src/admin/update_sol_usdc_rate.rs	39
programs/king-arthur-sc/src/admin/update_winner_percentage.rs	46
programs/king-arthur-sc/src/admin/update_withdraw_destination.rs	23
programs/king-arthur-sc/src/admin/withdraw_treasury.rs	72
programs/king-arthur-sc/src/context/initialize.rs	71
programs/king-arthur-sc/src/context/mod.rs	3
programs/king-arthur-sc/src/state/admin_transfer.rs	13
programs/king-arthur-sc/src/state/config.rs	43
programs/king-arthur-sc/src/state/error.rs	96
programs/king-arthur-sc/src/state/mod.rs	13
programs/king-arthur-sc/src/state/package.rs	50
programs/king-arthur-sc/src/state/winner.rs	13
programs/king-arthur-sc/src/state/winner_wallet.rs	7
programs/king-arthur-sc/src/user/buy_package.rs	107
programs/king-arthur-sc/src/user/claim_prize.rs	65
programs/king-arthur-sc/src/user/mod.rs	5
programs/king-arthur-sc/src/utils/mod.rs	3
programs/king-arthur-sc/src/utils/sigmoid.rs	208
programs/king-arthur-sc/src/lib.rs	159
Total	1992

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **High** issues: **1**
- **Medium** issues: **2**
- **Info** issues: **3**

ID	Title	Severity	Status
[H-01]	Pre-funding PDA Griefing Attack	High	Fixed
[M-01]	Irreversible <code>total_sold</code> Counter and Missing On-Chain User State Corrupts Dynamic Pricing Model	Medium	Fixed
[M-02]	Inconsistent State Leading to DoS	Medium	Acknowledged
[I-01]	Same Wallet Can Win Multiple Times in the Same Round After Claiming	Info	Acknowledged
[I-02]	Unrestricted Prize & Treasury Split Manipulation During Active Game	Info	Acknowledged
[I-03]	Inconsistent Winner Distribution Due to Mid-Round Winner Percentage Changes (Centralization Risk)	Info	Acknowledged

7. Findings

[H-01] Pre-funding PDA Griefing Attack

Severity

High Risk

Description

The `set_winner()` in `set_winner.rs` instruction is vulnerable to a PDA Initialization Griefing attack (also known as Pre-funding DoS on PDA Creation).

The `winner_wallet` PDA is created using Anchor's `init` constraint with deterministic, predictable seeds: `[b"winner_wallet", winner_address.as_ref()]`. Since PDA addresses are derived deterministically from these seeds and the program ID, any party can compute the exact PDA address for any potential winner off-chain.

An attacker can exploit this by transferring a minimal amount of SOL (as little as 1 lamport) to the target PDA address before the `prize_admin` calls `set_winner()`.

Anchor's `init` constraint relies on the System Program's `create_account()` instruction, which requires the target address to have **zero lamports and zero data length**. Pre-funding the PDA violates this requirement, causing the `set_winner()` transaction to fail permanently with an **"account already in use"** error.

This attack is particularly severe in the context of this gaming/lottery application, where: - Participants know each other and their public keys are public knowledge - Competitors have a direct financial incentive to grief rivals - The attack cost is negligible (1 lamport \$0.00000001)

Location of Affected Code

File: [programs/king-arthur-sc/src/admin/set_winner.rs#L49-L56](#)

```
#[account(
  init,
  payer = prize_admin,
  space = 8 + WinnerWallet::INIT_SPACE,
  seeds = [b"winner_wallet", winner_address.as_ref()],
  bump
)]
pub winner_wallet: Account<'info, WinnerWallet>,
```

Impact

1. **Permanent Winner Exclusion:** A targeted wallet address can **never** be set as a winner. The attack is irreversible with the current program design and the cost of the attack is very LOW.
2. **No Recovery Mechanism:**
 - Lamports sent to the grieved PDA are permanently stuck (no one can sign for a PDA)
 - No program instruction exists to drain or reclaim the grieved PDA
 - The victim must abandon their wallet entirely and use a new address
3. **Competitive Sabotage:** In a gaming context, malicious players can systematically exclude competitors from prize eligibility at near-zero cost.
4. **Scalable Attack:** An attacker can pre-grief multiple potential winners by deriving and funding their respective PDAs in advance.

Proof of Concept

Attack Flow: 1. Attacker identifies victim's public key (trivial in gaming context) 2. Attacker derives `winner_wallet` PDA using known seeds 3. Attacker transfers 1 lamport to the PDA address 4. `prize_admin` attempts `set_winner(victim_address)` 5. Transaction fails — victim permanently excluded from winning

Recommendation

Consider applying one of the following: - Replace the `init` constraint with `init_if_needed` combined with explicit validation to handle the pre-funded account case - Add validation in the handler to detect grieving attempts - Use a nonce or timestamp in the PDA seeds to make them unpredictable

Team Response

Fixed.

[M-01] Irreversible `total_sold` Counter and Missing On-Chain User State Corrupts Dynamic Pricing Model

Severity

Medium Risk

Description

The `buy_package()` instruction in the King Arthur smart contract lacks on-chain user state tracking and provides no mechanism to correct the global `total_sold` counter. When a user purchases a package, the contract:

1. Calculates dynamic pricing based on `total_sold`
2. Transfers SOL to `prize_pool` and `treasury` PDAs
3. Increments `config.total_sold`
4. Emits a `PackagePurchased` event

Critical Gap: The user receives **no on-chain record** of their purchase. No user-specific PDA is created, no token is minted, and no credits are stored on-chain. The `BuyPackageEvent` struct is used solely for event emission:

```
#[derive(Clone, AnchorSerialize, AnchorDeserialize)]
pub struct BuyPackageEvent {
    pub user: Pubkey,
    pub package_id: u64,
    pub credits: u64,
}
```

The entire credit tracking system relies on off-chain indexers parsing emitted events. This creates a fragile dependency where:

- If indexers experience downtime, data loss, or bugs, user credits are irrecoverable
- The `total_sold` counter is **monotonically increasing** with no admin function to decrement or correct it
- Failed or erroneous transactions that increment `total_sold` cannot be reversed

Location of Affected Code

File: [programs/king-arthur-sc/src/user/buy_package.rs#L88-L96](#)

```
self.config.total_sold += 1;

emit!(PackagePurchased {
    user: self.user.key(),
    package_id: self.package.id,
    price_usdc: current_price_usdc,
    price_lamports,
    total_sold: self.config.total_sold,
});
```

File: [programs/king-arthur-sc/src/state/config.rs#L42](#)

```
pub total_sold: u64,    // Global package purchase counter (used for
                        sigmoid pricing)
```


Impact

1. **Data Integrity Risk:** Off-chain indexer failures result in permanent loss of user credit records with no on-chain source of truth for recovery.
2. **No Error Correction:** If a transaction incorrectly increments `total_sold` (e.g., due to a bug, replay, or operational error), there is no admin instruction to decrement or adjust the counter.
3. **Pricing Drift:** Since `total_sold` drives the sigmoid dynamic pricing curve, an incorrect counter permanently skews all future package prices — either too high (if over-counted) or too low (if under-counted upon recovery).
4. **Audit Trail Gap:** On-chain state provides an immutable audit trail. Relying solely on events means the audit trail exists only in indexer databases, which may be mutable, lossy, or disputed.
5. **On-Chain Price Calculation Corruption:** The `total_sold` counter is a **critical input to the on-chain sigmoid pricing algorithm**. Every `buy_package()` call uses `total_sold` to compute the current price via `calculate_current_price(config.total_sold, ...)`. An incorrect or inflated `total_sold` value directly corrupts the price calculation for **all future purchases**, causing users to pay incorrect prices. This is not merely an off-chain data issue — it fundamentally breaks the on-chain economic model.

Recommendation

Consider the following solutions: 1. Add On-Chain User State Tracking 2. Add Admin Correction Function

Team Response

Fixed.

[M-02] Inconsistent State Leading to DoS

Severity

Medium Risk

Description

The `cancel_winner()` instruction fails to recalculate the `is_game_stopped` flag after decrementing `active_winner_count`. This creates a state inconsistency where the game remains stopped even when there is capacity for additional winners.

When a winner is set and `active_winner_count` reaches `max_winners`, the `is_game_stopped` flag is set to `true` in `set_winner()`. However, when a winner is subsequently cancelled via `cancel_winner()`, the `active_winner_count` is decremented, but `is_game_stopped` is **never recalculated**. This leaves the contract in an invalid state where:

- `active_winner_count < max_winners` (there's room for more winners)
- `is_game_stopped = true` (but the game is blocked)

Both `buy_package()` and `set_winner()` have constraints that check `!config.is_game_stopped`, meaning: - No new packages can be purchased - No new winners can be set - Prize pool funds become locked

The only recovery mechanism is for the admin to call `update_max_winners()`, which happens to recalculate `is_game_stopped`. However, this is a non-obvious workaround that requires admin awareness of the bug.

Location of Affected Code

File: [programs/king-arthur-sc/src/admin/cancel_winner.rs#L106-L107](#)

```
pub fn cancel_winner(&mut self, winner_id: u64, remaining_accounts: &[
    AccountInfo<'info>]) -> Result<()> {
    // code

    self.winner.is_evaluated = true;
    self.config.active_winner_count -= 1;
    // BUG: is_game_stopped is NOT recalculated here

    // code
}
```

The flag is set in `set_winner.rs` but never cleared in `cancel_winner.rs`:

File: [programs/king-arthur-sc/src/admin/set_winner.rs#L120-L121](#)

```
pub fn set_winner(&mut self, winner_address: Pubkey) -> Result<()> {
    // code
    if self.config.active_winner_count == self.config.max_winners {
        self.config.is_game_stopped = true;
        emit!(GameStopped {
            final_winner_count: self.config.active_winner_count,
            stopped_by: self.prize_admin.key(),
        });
    }
    // code
}
```

Constraints that block operations when `is_game_stopped` is true:

File: [programs/king-arthur-sc/src/user/buy_package.rs#L26](#)

```
constraint = !config.is_game_stopped @ Errors::GameStopped,
```

File: [programs/king-arthur-sc/src/admin/set_winner.rs#L36](#)

```
constraint = !config.is_game_stopped @ Errors::GameStopped,
```

Impact

1. **Fund Lock:** Prize pool funds become inaccessible for new winner allocations. Existing funds in the prize pool cannot be distributed to new winners until admin intervention.
2. **Protocol DoS:** The entire game halts unexpectedly. Users cannot purchase packages, and the prize admin cannot set new winners, even though the system should have capacity for more winners.

3. **Silent Failure:** There is no event or error indicating the state inconsistency. The admin may not realize the game is stuck until users report issues.
4. **Non-Obvious Recovery:** The only workaround (`update_max_winners`) is not intuitive. An admin would not naturally think to “update” `max_winners` to the same value just to unstick the game.
5. **Economic Impact:** During the stuck period, no purchases occur, reducing prize pool growth and treasury revenue.

Proof of Concept

Attack Scenario:

- `max_winners` = 3, winners 0, 1, 2 are set
- `is_game_stopped = true` (automatically set when `active=3=max`)
- Admin cancels winner 0 `active_winner_count` = 2
- `is_game_stopped` is still true
- Game is stuck: No new winners can be set, no packages can be purchased
- Prize pool funds are locked til Admin intervenes

Recommendation

It is recommended to add `is_game_stopped` recalculation in `cancel_winner()` after decrementing `active_winner_count` which ensures the flag accurately reflects the current state after any winner count change.

Team Response

Acknowledged. The current implementation of the `is_game_stopped` flag is a deliberate design choice intended to prioritize system stability over automation. We believe that automatically toggling the game state back to ‘active’ can create operational friction and unexpected state transitions. Since users retain their credits for future rounds and the prize pool remains securely locked and untouched during this period, we view the manual reset as a necessary ‘safety pause’ that allows for administrative verification before the game resumes.

[I-01] Same Wallet Can Win Multiple Times in the Same Round After Claiming

Severity

Informational Risk

Description

The contract allows the same wallet to be set as a winner more than once in a single round. This contradicts the documented intent in `claim_prize.rs`.

According to the comment – the `claim_prize()` closes `winner_wallet` “so wallet can win again in **future rounds**”

Current Behaviour After a winner claims, their `winner_wallet` PDA is closed. `set_winner()` uses `init` for `winner_wallet` with seeds `[b"winner_wallet", winner_address]`. Because the account no longer exists, `init` succeeds and the same `winner_address` can be set as a winner again **in the same round**.

There is no check that `winner_address` has not already won in the current round. Although only `prize_admin` can call `set_winner()`, only a trusted actor can cause this inconsistency.

Location of Affected Code

File: [programs/king-arthur-sc/src/user/claim_prize.rs#L22-L28](#)

```
/// WinnerWallet account - maps wallet address to winner ID
/// Closed on claim so wallet can win again in future rounds
#[account(
    mut,
    seeds = [b"winner_wallet", wallet.key().as_ref()],
    bump,
    close = payer
)]
pub winner_wallet: Account<'info, WinnerWallet>,
```

File: [programs/king-arthur-sc/src/admin/set_winner.rs#L49-L56](#)

```
#[account(
    init,
    payer = prize_admin,
    space = 8 + WinnerWallet::INIT_SPACE,
    seeds = [b"winner_wallet", winner_address.as_ref()],
    bump
)]
pub winner_wallet: Account<'info, WinnerWallet>,
```

File: [programs/king-arthur-sc/src/admin/set_winner.rs#L72-L130](#)

```
pub fn set_winner(&mut self, winner_address: Pubkey) -> Result<()> {
    // No check: winner_address already has active winner this round
    let prize_pool_balance = self.prize_pool.to_account_info().lamports()
    ;
    // code
}
```

File: [programs/king-arthur-sc/src/state/winner_wallet.rs](#)

```
#[account]
#[derive(InitSpace, Default)]
pub struct WinnerWallet {
    pub winner_id: u64, // Maps wallet address to winner ID only; no
        round_id
}
```

Impact

`max_winners` can be filled with fewer than `max_winners` unique wallets, so “winners per round” is no longer “unique wallets per round”.

Proof of Concept

Scenario: Round with `max_winners = 3`; same wallet wins twice in the same round.

1. `set_winner(alice)` creates `winner` (`id=0`), `winner_wallet(alice)` (`winner_id=0`).
2. `accept_winner(0)`.
3. `claim_prize()` (Alice) prize transferred, `winner` and `winner_wallet(alice)` **closed**.
4. `set_winner(alice)` again `winner_wallet(alice)` PDA does not exist, so `init` succeeds new `winner` (`id=1`), new `winner_wallet(alice)` (`winner_id=1`).
5. `set_winner(bob)` winner `id=2`.

Result: Round has 3 winner slots but only 2 unique wallets (Alice twice, Bob once). Alice wins twice in the same round.

Recommendation

Make `winner_wallet` PDA round scoped with `config.round_start_winner_id` and do not close it on claim, so the `same wallet cannot be set as a winner twice in the same round` and can still win again in a future round.

Team Response

Acknowledged.

[I-02] Unrestricted Prize & Treasury Split Manipulation During Active Game

Severity

Informational Risk

Description

The `update_percentages()` instruction allows the admin to modify `prize_percentage` and `treasury_percentage` at any time during an active game round without any constraints on minimum/maximum values or game state checks. The only validation is that both percentages must be greater than zero and sum to 1000 basis points (100%).

When users purchase packages via `buy_package()`, the funds are split between the prize pool and treasury using the current `config.prize_percentage` value at the time of their purchase. If the admin changes these percentages mid-round, different users purchasing in the same round will contribute drastically different amounts to the prize pool versus the treasury.

For example: - User A buys a package when `prize_percentage = 950` [95%] 95% goes to prize pool - Admin calls `update_percentages(500, 500)` - User B buys a package when `prize_percentage = 500` [50%] only 50% goes to the prize pool

Location of Affected Code

File: [programs/king-arthur-sc/src/admin/update_percentages.rs#L17-L37](#)

```
impl<'info> UpdatePercentages<'info> {
    pub fn update_percentages(
        &mut self,
        prize_percentage: u16,
        treasury_percentage: u16,
    ) -> Result<()> {
        require!(
            prize_percentage + treasury_percentage == BASIS_POINTS_TOTAL,
            Errors::InvalidPercentages
        );
        require!(
            prize_percentage > 0 && treasury_percentage > 0,
            Errors::InvalidPercentages
        );

        self.config.prize_percentage = prize_percentage;
        self.config.treasury_percentage = treasury_percentage;

        Ok(())
    }
}
```

File: [programs/king-arthur-sc/src/user/buy_package.rs#L66-L68](#)

```
let prize_pool_share =
    ((price_lamports as u128 * self.config.prize_percentage as u128) /
     1000) as u64;
let treasury_share = price_lamports - prize_pool_share;
```

Account Constraints: [programs/king-arthur-sc/src/admin/update_percentages.rs](#)

```
#[account(
    mut,
    has_one = admin @ Errors::Unauthorized
)]
pub config: Account<'info, Config>,
```

There is no `!config.is_game_stopped` or `!config.is_finalized` constraint.

Impact

1. Users purchasing packages in the same round receive vastly different prize pool contributions based on timing, with no transparency or warning about parameter changes.

2. Admin can effectively drain prize pool growth by setting an extremely low `prize_percentage` (1 basis point = 0.1%) while routing 99.9% to treasury, reducing the total prize amount available for winners.

Proof of Concept

1. Game initializes with `prize_percentage = 950` (95%), `treasury_percentage = 50` (5%)
2. 100 users buy packages at 1 SOL each 95 SOL goes to prize pool
3. Admin calls `update_percentages(1, 999)` prize=0.1%, treasury=99.9%
4. Next 100 users buy packages at 1 SOL each only 0.1 SOL goes to prize pool (99.9 SOL to treasury)
5. Prize pool grows by only 0.1 SOL instead of the expected 95 SOL for the second batch
6. Winners draw from a prize pool of 95.1 SOL instead of 190 SOL
7. Users in the second batch contributed almost nothing to prizes despite paying the same price

Recommendation

Add game state constraints to prevent mid-game changes

```
#[account(  
  mut,  
  has_one = admin @ Errors::Unauthorized,  
  constraint = !config.is_game_stopped @ Errors::  
    CannotUpdateDuringActiveGame,  
  constraint = config.active_winner_count == 0 @ Errors::  
    CannotUpdateWithActiveWinners,  
)]  
pub config: Account<'info, Config>,
```

Add minimum & maximum bounds

Enforce reasonable bounds to prevent extreme manipulation:

```
pub const MIN_PRIZE_PERCENTAGE: u16 = 500; // Minimum 50%  
pub const MAX_TREASURY_PERCENTAGE: u16 = 500; // Maximum 50%  
  
require!(  
  prize_percentage >= MIN_PRIZE_PERCENTAGE,  
  Errors::PrizePercentageTooLow  
);  
require!(  
  treasury_percentage <= MAX_TREASURY_PERCENTAGE,  
  Errors::TreasuryPercentageTooHigh  
);
```

Team Response

Acknowledged. The flexibility of the prize and treasury split is a core component of our dynamic economic model. As the project evolves, the team requires the agility to adjust these ratios to ensure long-term protocol sustainability and to respond to shifting market conditions. While we understand the transparency concerns, these adjustments are viewed as strategic financial decisions. We remain committed to balancing attractive user rewards with the necessary treasury growth required to scale the Stormrae ecosystem.

[I-03] Inconsistent Winner Distribution Due to Mid-Round Winner Percentage Changes (Centralization Risk)

Severity

Informational Risk

Description

- The `update_winner_percentage()` instruction allows the admin to modify the `winner_percentage` parameter at any time during an active game round, with no constraints on game state or reasonable bounds (only requires > 0 and 1000 basis points).
- As `set_winner()` is called, the prize amount is calculated as `winner_percentage` of the current `prize_pool_balance`, and this percentage is stored on the Winner account. The protocol allows the admin to change `winner_percentage` between setting different winners in the same round, in that case, each winner receives a different share of the prize pool, creating an inconsistent and unfair distribution.

Moreover, this violates the business logic as well, as mentioned in the `cancel_winner.rs`, which clearly states – `This ensures redistribution acts "like the cancelled winner never existed"`

Location of Affected Code

File: `programs/king-arthur-sc/src/admin/update_winner_percentage.rs`

```
impl<'info> UpdateWinnerPercentage<'info> {
    pub fn update_winner_percentage(&mut self, new_percentage: u16) ->
        Result<()> {
        require!(
            new_percentage > 0 && new_percentage <= BASIS_POINTS_TOTAL,
            Errors::InvalidWinnerPercentage
        );

        let old_percentage = self.config.winner_percentage;
        self.config.winner_percentage = new_percentage;

        emit!(WinnerPercentageUpdated {
            old_percentage,
            new_percentage,
        });

        Ok(())
    }
}
```

Impact

Violation of Business Logic: When a winner is cancelled via `cancel_winner()`, their prize amount is redistributed using the same X% geometric decay pattern. This ensures redistribution acts “like the cancelled winner never existed”

Recommendation

It is recommended that the winner percentage should be locked til a round completes.

Team Response

Acknowledged.

our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Thank you!

